MAREVLO · PRODUCTION RAG

Loading & Chunking

How to cut your documents into the right pieces — the single decision that matters most.

PART I — FOUNDATIONS

Module 2

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

Before your system can ever answer a question, it has to do some homework. It has to read your documents, cut them into smaller pieces, and store those pieces so they can be found later. This homework happens once, ahead of time, and it is called the **indexing pipeline**. This module covers the first two steps of that pipeline: **loading** the documents and **chunking** them into pieces.

Here is the most important idea in the whole module, and we will keep coming back to it: **the way you cut your documents into pieces decides, more than anything else, how good your answers will be.** Not the fancy model. Not the expensive database. The chunking. If you only have time to get one thing right in this entire course, get chunking right. We will explain exactly why, step by step, and we will show you the most common ways it goes wrong so you can avoid them before they ever happen.

> **THE WHOLE MODULE IN ONE SENTENCE**
>
> Good answers come from good pieces. If you hand your system messy or badly-cut pieces, no amount of cleverness later can rescue it — so we spend our care here, at the start, where it pays off the most.

A note on the code in this module

Every code example here is written from scratch around one running example of our own: a short, made-up appliance manual for a fictional coffee machine. We use the same example throughout so you can watch one document travel through the whole pipeline. The code uses real, public libraries — there is only one correct way to call a library, the same way there is only one way to write `2 + 2` — but the examples, the helper functions, and the lessons around them are ours. Library names and import paths change between versions, so always check them against the version you actually install rather than trusting any single source, including this page.

Where we are in the bigger picture

Recall from Module 1 that RAG has two pipelines. The **indexing pipeline** runs ahead of time and prepares your documents. The **query pipeline** runs every time a user asks something. This module lives entirely inside the indexing pipeline, at the very beginning:

```
# The indexing pipeline (runs once, ahead of time):
Raw documents  -->        -->                          -->  Embed  -->  Store

# This module covers the two bold steps. Embedding is Module 3.
```

SECTION 1

Loading: getting the text out of your files

Before you can cut a document into pieces, you have to read it in. That is the loader's job — and it has one trap that catches almost everyone.

A **loader** is a small piece of code that opens a file and pulls the text out of it. There is a different loader for each kind of source, because a PDF, a web page, and a plain text file all store their text in different ways. You point a loader at

your file, and it hands you back the text wrapped in a small object that also carries some extra information about where the text came from.

Let us write a tiny, original helper of our own that loads a file and then immediately tells us what we got back. The "tell us what we got back" part is the important habit — most loading bugs are invisible until you actually look at the result, so we build the looking right into the helper.

LAB 2.1 — Load a file and inspect what came back (our own helper)

```
# We import a PDF loader from a public library. Verify the exact import
# path against the version you installed -- these move between releases.
from langchain_community.document_loaders import PyPDFLoader


def load_and_describe(file_path):
    """Load a file and print a short report on what we received.

    The whole point of this helper is the *report*. Loading without
    looking is how silent bugs sneak in, so we make looking automatic.
    """
    pieces = PyPDFLoader(file_path).load()      # read the file in

    print(f"Loaded {len(pieces)} piece(s) from {file_path}")
    for index, piece in enumerate(pieces[:3]):  # peek at the first few
        text = piece.page_content
        print(f"  piece {index}: {len(text)} characters")
        print(f"    starts with: {text[:60]!r}")
        print(f"    came from:   {piece.metadata}")  # where it came from

    return pieces


manual_pieces = load_and_describe("coffee_machine_manual.pdf")
```

Notice the two parts of each loaded piece. There is the actual text, which we read with `.page_content`, and there is a small bundle of extra facts called `.metadata`, which might say which file and which page the text came from. That metadata looks unimportant now, but hold onto it: in later modules it becomes one of your most powerful tools, because it is how you will later tell the system things like "only search the 2025 documents" or "only this customer's files." It rides along with every piece, for free, so the rule is simple — never throw it away.

The one loading trap that catches everyone

Here is the trap. Different loaders hand you text in different sizes. A PDF loader usually gives you **one piece per page**. A web-page loader gives you **one piece per web page**. A loader for a chat export might give you **one piece per message**. If you quietly assume every piece is small and tidy, but your loader is actually handing you whole 40-page chapters as single pieces, then everything you do next behaves in ways you did not expect — and you will not know why, because the bug is upstream and invisible.

> **PLAIN RULE**

> After loading, always look. Print the number of pieces and the length of a few of them, exactly like our helper does. Two minutes of looking here will save you hours of confusion later, because bad loading quietly poisons every step that follows it.

SECTION 2

Why we cut documents up at all

It feels wasteful to slice a clean document into pieces. Here is why it is necessary — and why the pieces must be just the right size.

A fair question to ask is: why not just feed the whole document to the system and let it find the answer itself? There are two reasons, and understanding them is the key to everything else in this module.

Reason one: there is a hard size limit

The language model can only read so much text at one time. There is a firm ceiling on how much you can hand it per question. If your document is a 300-page manual, it simply will not fit. So you have to find the few pieces that are actually relevant and pass only those. But to find pieces, you must first *have* pieces — and that means cutting the document up ahead of time.

Reason two: small pieces are far easier to find (this is the big one)

To understand this, you need one idea from the next module, in its simplest possible form. In RAG, every piece of text gets turned into a kind of "meaning fingerprint" — a compact summary of what that piece is about. When a user asks a question, we turn the question into a fingerprint too, and then we hunt for the pieces whose fingerprint is the closest match. That matching is how the system finds the relevant text.

ANALOGY — THE MEANING FINGERPRINT

Think of the fingerprint as a single label that captures the gist of a piece of text. If a piece is short and focused — say, only about how to descale the machine — its label is sharp and clear: "this is about descaling." When someone asks how to descale, that label lights up as a strong match.

Now here is the problem with big pieces. If one piece talks about ten different things — descaling, water filters, warranty terms, error codes, cup sizes, and more — what is its single label? It collapses into something vague like "a bit of everything." And a label that means "a bit of everything" is a weak match for *every* question and a strong match for *none*. So the right piece never clearly wins, and the system either retrieves the wrong thing or retrieves nothing useful.

ANALOGY — THE BLURRY PHOTO

Embedding a huge chunk is like trying to describe an entire photo album in one sentence. The sentence comes out so general that it could match almost any search, which means it stands out for nothing in particular. A single sharp photo, by contrast, is easy to find when you search for exactly what is in it.

So we cut documents into pieces small enough that each piece is about roughly one thing. Then each piece earns a sharp, clear fingerprint, and the right piece becomes easy to find when its topic comes up. **That is the entire reason chunking matters so much.** But notice the balance hiding inside that sentence: cut too big, and every piece becomes a blurry "bit of everything" that is hard to find; cut too small, and a piece loses the surrounding context it needs to

make sense on its own. Finding the balance between those two failures is the whole craft of chunking, and it is what the rest of this module teaches you to do.

SECTION 3

Three ways to cut: from crude to careful

There are three common chunking strategies, going from simple-but-rough to smart-but-more-work. Most projects should use the middle one.

Way 1 — Fixed-size chunking (the blunt knife)

The simplest method is to cut a new piece every fixed number of characters. Every 500 characters, snip. It is fast and easy to understand. The trouble is that it cuts blindly: it does not care whether character number 500 happens to land in the middle of a sentence, the middle of a word, or the middle of a number. It very often does, and the result is broken, half-formed pieces. Here is our own minimal version so you can see exactly how blunt it is.

LAB 2.2 — Fixed-size cutting, written by hand (so you can see the bluntness)

```
def cut_fixed(text, size=500):
    """Cut text into equal-size pieces, ignoring sentence boundaries.

    This is deliberately naive. We are writing it out so you can *see*
    that it slices wherever the counter lands -- mid-word if need be.
    """
    pieces = []
    start = 0
    while start < len(text):
        pieces.append(text[start : start + size])  # blind slice
        start += size
    return pieces


rough_pieces = cut_fixed(manual_text, size=500)
print(f"Fixed cutting made {len(rough_pieces)} pieces")
```

Use this only for quick experiments, or for text that has no real structure to respect. For real documents, it is usually too crude, and the next method fixes its biggest flaw.

Way 2 — Recursive chunking (the sensible default)

This is the method you should reach for first in almost every project. Instead of cutting blindly at a fixed spot, it tries to cut at *natural* boundaries. It works in order of preference: first it tries to split between paragraphs, which is the cleanest possible cut; only if a paragraph is still too big does it fall back to splitting between sentences, then between words, and finally — as a last resort — mid-word. The word "recursive" simply means it keeps trying gentler cuts before resorting to harsher ones.

ANALOGY — SLICING A LOAF OF BREAD

Fixed-size chunking is like chopping a loaf every five centimetres even if that means cutting straight through a slice you wanted whole. Recursive chunking is like slicing along the natural marks first, so each slice stays in one clean piece. Same loaf, much tidier slices.

LAB 2.3 — Recursive chunking with a public splitter (our own wrapper)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter


def build_recursive_splitter(size=1000, overlap=150):
    """Create a recursive splitter with sensible, explained settings.

    We wrap the library call in our own function so the *reasoning*
    lives next to the settings, and so we can reuse it everywhere.
    """
    return RecursiveCharacterTextSplitter(
        chunk_size=size,        # aim for about this many characters
        chunk_overlap=overlap,  # repeat this much between pieces (Section 4)
        # Try these split points in order: blank line (paragraph),
        # then single line, then sentence, then space, then -- last
        # resort -- a hard character cut.
        separators=["\n\n", "\n", ". ", " ", ""],
    )


splitter = build_recursive_splitter(size=1000, overlap=150)

# split_documents keeps the metadata attached to each new piece,
# which is exactly why we feed it documents, not raw strings.
clean_pieces = splitter.split_documents(manual_pieces)
print(f"Recursive cutting made {len(clean_pieces)} pieces")
```

Way 3 — Structure-aware chunking (the careful, tailored cut)

Some documents come with a clear built-in structure: a Markdown file has headings, a code file has functions, a well-formed web page has sections. For these, the smartest move is to cut along that structure — one piece per section, one piece per function — so that each piece is a complete, self-contained unit of meaning. This takes a little more setup, but for structured content it produces the cleanest pieces of all, because the document is literally telling you where its natural boundaries are. Our own example below splits a small Markdown manual on its headings.

LAB 2.4 — Structure-aware splitting on Markdown headings (our own example)

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

# Our own tiny Markdown manual, written for this example.
markdown_manual = """
# Coffee Machine Manual

## Daily Cleaning
Rinse the drip tray and wipe the steam wand after each use.

## Descaling
Run a descaling cycle every two months using the descaling solution.
"""

# Tell the splitter which heading levels mark a new section.
section_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "title"), ("##", "section")],
)

# Each piece now lines up with a real section of the document,
# and the heading is saved into the piece's metadata for free.
sections = section_splitter.split_text(markdown_manual)
for piece in sections:
    print(piece.metadata, "->", piece.page_content[:40])
```

The simple way to choose between these three: if your documents have obvious structure, cut along it. If they are plain flowing text, recursive chunking is your best friend. And only fall back to fixed-size cutting for throwaway experiments. We will give you a clear decision guide at the end of the module.

SECTION 4

The two dials that actually matter

Whatever method you use, two settings control the outcome: chunk size and chunk overlap. Learn these two and you understand most of chunking.

Dial 1 — Chunk size: how big is each piece?

This is the most important setting you will ever touch in chunking. We met the trade-off in Section 2; now let us make it fully concrete with our running example.

If pieces are too big: each one covers too many topics, so its fingerprint becomes a blurry "bit of everything," and the right piece becomes hard to find. You also waste space when you finally pass the piece to the model, and you pay more money for those wasted characters.

If pieces are too small: each one loses the context around it. Imagine a piece that just says, *"Run it every two months."* Run *what* every two months? The descaling cycle — but that subject was in the sentence before, and that sentence got cut into a different piece. The piece is findable but useless on its own, because it cannot answer anything by itself.

> **THE HONEST TRUTH ABOUT CHUNK SIZE**
>
> There is no single correct number. The right size depends on your documents. Dense, fact-packed text — a contract, a technical spec, a settings table — wants smaller pieces, because a great deal of meaning is packed into very little text. Flowing, story-like text tolerates bigger pieces. A common *starting point* is somewhere around 500 to 1,500 characters, but treat that as a place to begin testing, never as a final answer. HEURISTIC

Dial 2 — Chunk overlap: stitching the seams

When you cut a document into pieces, an important fact will sometimes land right on the cut line — half in one piece, half in the next. To stop that fact from being lost, we let neighbouring pieces **overlap**: we repeat a small amount of text from the end of one piece at the start of the next. That way, if a sentence straddles the boundary, it survives whole in at least one of the two pieces.

> **ANALOGY — THE "PREVIOUSLY ON..." RECAP**
>
> Overlap is like the short recap at the start of a TV episode. It repeats the last moment of the previous episode so you do not lose the thread. The pieces still stand on their own, but the seam between them is no longer a place where meaning quietly falls through the cracks.

The cost of overlap is small — a little extra storage and a little repeated text — and the benefit is that you stop losing facts at the seams. A modest overlap of roughly 10 to 20 percent of your chunk size is a reasonable place to start. HEURISTIC But overlap also has a sharp edge that beginners cut themselves on, which brings us to our first fault.

SECTION 5

Faults and corrections: the bugs you will actually hit

Below are six real chunking faults. For each one we show the broken code, explain plainly why it breaks, and then show the corrected version. Reading these now saves you from discovering them the hard way in production.

FAULT 1 — Overlap as big as the chunk itself

THE BROKEN CODE

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=300,
    chunk_overlap=300,   # overlap equals the chunk size
)
```

WHY IT BREAKS

Overlap is supposed to be a *small* repeat between neighbours. If the overlap is as large as the chunk (or larger), each new piece is almost entirely a copy of the previous one. You end up with a flood of near-duplicate pieces, your storage and embedding costs balloon, and retrieval gets crowded with copies of the same text instead of variety. Some versions will also simply refuse and raise an error.

THE CORRECTION

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=300,
    chunk_overlap=45,    # ~15% of the chunk size -- a small, safe seam
)
```

Keep overlap to a fraction of the chunk size. A good rule of thumb is 10–20%.

FAULT 2 — Cutting raw strings, so the metadata is thrown away

THE BROKEN CODE

```
# Pulling out just the text loses where it came from.
raw_text = manual_pieces[0].page_content
pieces = splitter.split_text(raw_text)   # returns plain strings, no metadata
```

WHY IT BREAKS

When you split raw text, the pieces come back as bare strings with no memory of which file or page they came from. You have silently discarded the metadata. Later, when you want to say "only search this customer's documents" or "show me the source of this answer," you cannot — the information is gone, and re-creating it means re-indexing everything.

THE CORRECTION

```
# Split the documents themselves; the metadata rides along.
pieces = splitter.split_documents(manual_pieces)
print(pieces[0].metadata)   # source and page are still attached
```

Prefer `split_documents` over `split_text` whenever you have real documents. Keep the metadata alive end to end.

FAULT 3 — Separators that never match your text, giving you one giant piece

THE BROKEN CODE

```
# This text uses Windows line endings (\r\n) and has no blank lines,
# but our separators only look for "\n\n". Nothing matches.
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    separators=["\n\n"],   # the only separator -- and it is never found
)
pieces = splitter.split_documents(manual_pieces)
# Result: huge pieces, because the splitter could not find a place to cut.
```

WHY IT BREAKS

A splitter can only cut where it finds a separator. If you give it separators that do not exist in your text — for example, looking for blank lines in text that has none, or ignoring the line-ending style your files actually use — it never finds a clean cut and falls back to producing oversized pieces. The fingerprints then go blurry (Fault style of Mistake 1), and you will not understand why until you inspect the pieces.

THE CORRECTION

```
# First, look at the text and normalise line endings; then give the
# splitter a realistic ladder of separators to try.
for piece in manual_pieces:
    piece.page_content = piece.page_content.replace("\r\n", "\n")

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=75,
    separators=["\n\n", "\n", ". ", " ", ""],  # gentle to harsh
)
pieces = splitter.split_documents(manual_pieces)
```

Always inspect a sample of the real text first, then choose separators that actually occur in it. Provide a ladder from gentle to harsh so there is always a fallback.

FAULT 4 — Measuring in characters when the real limit is tokens

THE BROKEN CODE

```
# We size pieces by character count, but the model counts in "tokens".
# For dense text, a 1000-character piece can be far more tokens than
# we assumed, quietly overflowing the budget later.
splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
```

WHY IT BREAKS

Models do not read characters; they read "tokens," which are small chunks of words. Some text packs many tokens into few characters (code, tables, languages without spaces). If you size by characters, your pieces can hold far more tokens than you expected, and when you assemble several of them for the model you blow past its limit and get silent truncation. The fix is to measure length the same way the model does.

THE CORRECTION

```
# Count length in tokens, using a tokenizer that matches your model.
# (Library names vary -- verify against your own stack.)
import tiktoken

encoder = tiktoken.get_encoding("cl100k_base")

def count_tokens(text):
    """Return how many tokens this text uses, the way the model sees it."""
    return len(encoder.encode(text))

# Tell the splitter to measure pieces in tokens, not characters.
splitter = RecursiveCharacterTextSplitter(
    chunk_size=250,            # now this means ~250 tokens
    chunk_overlap=40,
    length_function=count_tokens,
)
```

When token budgets matter — and in production they always do — measure chunk size in tokens, not characters.

FAULT 5 — Chunking the junk along with the text

THE BROKEN CODE

```
# Straight from loader to splitter, with no cleaning in between.
# Page headers, footers, and menu text get chunked as if they were content.
pieces = splitter.split_documents(manual_pieces)
```

WHY IT BREAKS

Loaders grab everything on a page, including repeated headers, footers, page numbers, navigation menus, and — for scanned documents — garbled text from imperfect reading. If you chunk that noise alongside the real content, the noise gets its own fingerprints and competes for retrieval slots. The cleanest model in the world cannot rescue an index full of cookie banners.

THE CORRECTION

```
# Clean first, then chunk. Our own small cleaner removes obvious junk.
import re

def clean_text(text):
    """Strip the most common boilerplate before chunking."""
    text = re.sub(r"Page \d+ of \d+", "", text)   # page numbers
    text = re.sub(r"\n{3,}", "\n\n", text)         # collapse blank gaps
    return text.strip()

for piece in manual_pieces:
    piece.page_content = clean_text(piece.page_content)

pieces = splitter.split_documents(manual_pieces)
```

Cleaning is a first-class step, not an afterthought. Clean, then chunk — never the other way around.

FAULT 6 — Building the index twice and doubling everything

THE BROKEN CODE

```
# Re-running the indexing script appends the same pieces again,
# so the store now holds two copies of every chunk.
for piece in pieces:
    store.add(piece)   # no check for "have I added this already?"
```

WHY IT BREAKS

Indexing feels like a one-time job, so people run the script again after a small change without clearing the old data. Now every chunk exists twice. Retrieval fills up with duplicates (so you get the same answer fragment four times instead of four different useful pieces), and you have paid to embed everything twice. This is one of the most common "why did it get worse?" surprises.

THE CORRECTION

```
# Give each piece a stable ID derived from its content, so re-adding
# the same piece overwrites rather than duplicates.
import hashlib

def stable_id(piece):
    """A repeatable ID: same text always produces the same ID."""
    return hashlib.sha256(piece.page_content.encode()).hexdigest()

for piece in pieces:
    store.add(piece, id=stable_id(piece))   # safe to re-run
```

Make indexing safe to re-run by giving each piece a stable identity, or by clearing the old data first on purpose. We return to this properly in the vector-database module.

SECTION 6

See it happen: the same text, cut three ways

Reading about chunking only gets you so far. Let us cut a real paragraph from our manual and look at the actual pieces it produces.

Here is a short passage from our fictional coffee-machine manual. We will cut it three different ways and look at exactly what each piece contains, so the difference between a good cut and a bad one stops being abstract.

THE ORIGINAL TEXT

"Fill the water tank to the marked line before brewing. Use filtered water for the best taste. The machine will beep twice when it is ready. To brew, press the large button once. For a double shot, press it twice within two seconds."

CUT 1 — FIXED-SIZE, 80 CHARACTERS, NO OVERLAP (the blunt knife)

PIECE 1

"Fill the water tank to the marked line before brewing. Use filtered water for the **be**"

PIECE 2

"**st taste.** The machine will beep twice when it is ready. To brew, press the large **bu**"

PIECE 3

"**tton once.** For a double shot, press it twice within two seconds."

What went wrong: the cuts land mid-word ("be / st taste", "bu / tton"). If someone asks "how do I make a double shot?", the instruction is intact in Piece 3, but the brewing instruction it depends on is broken across Pieces 2 and 3. The pieces read like a torn page.

CUT 2 — RECURSIVE, ~120 CHARACTERS, NO OVERLAP (natural boundaries)

PIECE 1

"Fill the water tank to the marked line before brewing. Use filtered water for the best taste."

PIECE 2

"The machine will beep twice when it is ready. To brew, press the large button once. For a double shot, press it twice within two seconds."

Much better: each piece is made of whole sentences. The brewing instructions now sit together and complete inside Piece 2. Each piece makes sense on its own. This is exactly why recursive chunking is the sensible default.

CUT 3 — RECURSIVE WITH OVERLAP (seams stitched)

PIECE 1

"Fill the water tank to the marked line before brewing. Use filtered water for the best taste."

PIECE 2

"Use filtered water for the best taste. The machine will beep twice when it is ready. To brew, press the large button once."

The highlighted sentence is repeated in both pieces. Now a question about water quality will match whichever piece is most relevant overall, and the fact is never stranded on a cut line. That repeated text is the overlap doing its quiet job.

Look at the gap between Cut 1 and Cut 2. Same text, same basic idea of "make smaller pieces," but the quality of the pieces is night and day. That gap *is* the difference between a RAG system that works and one that quietly frustrates everyone who uses it. And notice we have not touched the model or the database yet — this is all decided purely by how we cut.

SECTION 7

A practical starting recipe

When you do not know where to begin, begin here, then adjust based on what you actually see in your pieces.

Chunking is something you tune by looking at results, not by finding a magic number. But staring at a blank page is hard, so here is a sensible place to start, depending on what your documents look like. Treat every number below as a starting point to test and adjust, never as a fixed rule. HEURISTIC GUIDANCE

If your documents are flowing prose (reports, articles, handbooks):
Start with recursive chunking, chunk size around 1,000 characters, overlap around 150–200.

If your documents are dense and fact-packed (contracts, technical specs, policies):
Start with recursive chunking but smaller — chunk size around 500–700 — because a lot of meaning is packed into little text. Consider measuring in tokens (Fault 4).

If your documents have clear structure (Markdown, code, well-formed HTML):
Use structure-aware chunking — cut on headings, sections, or functions so each piece is a complete unit.

If your documents contain tables or lists:
Handle these specially — keep tables together with their headers; never let a blind cut separate a number from the label that explains it.

Always, before you trust your chunks:
Run the eyeball test (next) — read several random pieces and ask, "Could I answer a question using only this piece?" If yes, your chunking is healthy. If no, adjust and try again.

LAB 2.5 — The eyeball test, as a tiny helper (our own)

```
import random

def eyeball_test(pieces, how_many=5):
    """Print a few random pieces so a human can judge them.

    No metric beats reading your own chunks. This helper just makes
    the habit easy: it pulls a random sample for you to inspect.
    """
    sample = random.sample(pieces, min(how_many, len(pieces)))
    for number, piece in enumerate(sample, start=1):
        print(f"\n--- sample {number} "
              f"({len(piece.page_content)} chars) ---")
        print(piece.page_content)
        print("Could you answer a question using ONLY this piece? (y/n)")


eyeball_test(clean_pieces, how_many=5)
```

THE EYEBALL TEST IS YOUR BEST TOOL

> No dashboard beats simply reading your own chunks. Pull ten at random and read them as if you were the system trying to answer from them. You will spot oversized, undersized, and broken pieces in about two minutes — faster and more reliably than any automated measure. Do this every single time you change a setting.

Checkpoint: try it yourself

The best way to learn chunking is to watch it happen on your own document. Here is a short exercise that takes about twenty minutes and will teach you more than re-reading this module would.

1. Take one document you care about — ideally a real PDF or text file from your own work — and load it with the `load_and_describe` helper from Lab 2.1 so you can see how your loader splits it.
2. Chunk it three ways using the helpers above: fixed-size at 500 characters; recursive at 1,000 with 200 overlap; and recursive at 500 with 50 overlap. Print how many pieces each setting produced.
3. Run the `eyeball_test` helper on each version, pulling five random pieces. For each piece, answer the test question honestly: could you answer a question using only this piece?
4. Write down which setting gave you the most pieces that passed the eyeball test. That setting is your starting point for this document — and notice it may differ from the recipe defaults, because your document is unique.

> **WHAT YOU SHOULD NOTICE**
>
> The fixed-size version will have pieces that start or end mid-sentence, and they often fail the eyeball test. The recursive versions will read more cleanly. And the best chunk size will depend on how dense your particular document is. There is no universal winner, and discovering that for yourself — on your own document — is the entire point of the exercise.

What to carry into Module 3

> **THE SIX THINGS TO REMEMBER**
>
> **One,** chunking is the highest-leverage decision in the whole pipeline — get it right before reaching for anything fancier. **Two,** we cut documents up so each piece earns a sharp, findable meaning fingerprint. **Three,** pieces that are too big go blurry and hard to find; pieces that are too small lose their context — your job is the balance between them. **Four,** recursive chunking is the sensible default; use structure-aware chunking when your documents have clear structure; use a small overlap to stitch the seams. **Five,** watch out for the six faults — oversized overlap, lost metadata, mismatched separators, character-vs-token confusion, chunking the junk, and double-indexing. **Six,** the eyeball test — reading your own chunks — beats every other way of checking your work.

In Module 3 we finally open up the "meaning fingerprint" we kept referring to. You will learn what it really is — it is called an embedding — why it lets a computer measure how similar two pieces of text are, and how to choose the tool that creates it. Everything you cut in this module is about to be turned into one of these fingerprints, which is exactly why the quality of your cutting decides the quality of your fingerprints.